on the gem dependencies. The only reason it's being rerun is because an earlier step was invalidated in our Dockerfile.

Let's see if there's something we can do about this problem.

The Gemfile Caching Trick

It turns out there's an effective way to prevent changes to unrelated files busting our cache and causing a rebuild of all our gems from scratch. The trick is to *separate the copying of files that should* trigger a rebuild of our gems from those that *shouldn't*.

Let's update our Dockerfile to do this:

```
Line 1 FROM ruby:2.6
-
- RUN apt-get update -yqq && apt-get install -yqq --no-install-recommends \
-   nodejs
5
- COPY Gemfile* /usr/src/app/
- WORKDIR /usr/src/app
- RUN bundle install
-
10 COPY . /usr/src/app/
-
- CMD ["bin/rails", "s", "-b", "0.0.0.0"]
```

The first three instructions remain the same, but line 6 is new. It copies our Gemfile and Gemfile.lock into our image *before* the rest of our code.

```
COPY Gemfile* /usr/src/app/
```

This creates a separate, independent layer. Docker's cache for this layer will only be busted if either of these two files change.

Having copied our Gemfile and Gemfile.lock into our image, we can now change into the directory where they are and install our gems:

```
WORKDIR /usr/src/app
RUN bundle install
```

Finally, with our gems installed, we can copy the remainder of our source files into the image:

```
COPY . /usr/src/app/
```

Now, changes to all remaining files copied in this step will only bust the cache at *this* instruction, which is *after* our gems have been installed—just what we want.

Now, let's rebuild our image using this updated Dockerfile. Remember, though, that we've now changed all except the first three instructions in the Dockerfile,